

ANÁLISIS TÉCNICO

Sistema Autónomo de Análisis de Memecoins en Solana

Blueprint v2.2 → Prompt Cline v2.3

Virtudes · Puntos de Fallo · Investigación Validada · Prompt Cline
Marzo 2026

1. VIRTUDES DEL BLUEPRINT v2.2

El blueprint v2.2 demuestra una madurez técnica significativa respecto a versiones anteriores. A continuación, las decisiones de diseño que deben preservarse intactas en v2.3:

#	Virtud	Por qué preservarla
V1	Retroanálisis first (90 días antes de producción)	Elimina el problema del cold-start. Solución correcta y validada empíricamente.
V2	Tres modelos separados (pump / rug / survival)	Confundir los tres en un solo label hace los modelos inútiles. Decisión crítica correcta.
V3	Split temporal estricto (nunca aleatorio)	La única forma de evitar data leakage en series temporales financieras.
V4	max_feature_window_minutes por modelo	Previene leakage de features futuras. Solución elegante y rara en proyectos amateur.
V5	feature_version + model_version en cada fila	Sin esto no hay reproducibilidad. Crucial para comparar experimentos.
V6	precision@top_k como métrica principal	Correcto para clases extremadamente desbalanceadas (0.63% graduation rate real).
V7	LLM solo offline semanal, nunca en ruta crítica	Evita latencia y coste en el loop de decisión. Arquitectura correcta.
V8	Defense first: rug detection > pump detection	El edge real está en preservar capital, no en encontrar pumps.
V9	ON CONFLICT DO NOTHING en todos los inserts	Backfill idempotente. Permite reanudar sin corrupción de datos.
V10	Docker Compose con healthchecks	Despliegue reproducible y fácilmente reiniciable por Cline.
V11	EXECUTION_MODE=research por defecto	El sistema nunca ejecuta trades sin activación explícita del usuario.
V12	Actualización bayesiana de hipótesis (Beta)	Más honesta que promedios simples. Correcto para señales débiles.

2. PUNTOS DE FALLO IDENTIFICADOS

2.1 CRÍTICOS — Rompen el sistema en producción

FALLO C1 — PumpPortal WebSocket ausente (más impactante)

El blueprint usa polling HTTP cada 15 minutos contra Helius RPC para detectar tokens nuevos. Esto es funcionalmente incorrecto para producción.

Existe una API oficial y gratuita de PumpPortal que entrega eventos en tiempo real vía WebSocket:

```
wss://pumpportal.fun/api/data -- método: subscribeNewToken
```

Con polling a 15 min, cuando el sistema detecta un token ya llevan 15 minutos de actividad. Las features de ventana 0-5 minutos (las más predictivas según la literatura) son irrelevantes para la decisión porque ya han pasado. Para el backfill histórico el polling es aceptable; para producción, el WebSocket es obligatorio.

Solución v2.3: `collect_onchain.py` debe tener dos modos: BACKFILL (polling HTTP) y LIVE (WebSocket asyncio persistente).

FALLO C2 — PumpSwap completamente ignorado

Desde marzo de 2025, el 95%+ de tokens que 'gradúan' de la bonding curve de Pump.fun migran a PumpSwap (el DEX propio de Pump.fun), NO a Raydium. El blueprint no menciona PumpSwap en ningún lugar.

Impacto directo: el `label_still_active_7d` no puede calcularse correctamente sin datos de PumpSwap. Un token que migra a PumpSwap aparecerá con `liquidez_pool = 0` en los endpoints de Pump.fun, haciendo que el `label_targets.py` lo marque incorrectamente como rug pull.

Solución v2.3: añadir detección de migración a PumpSwap como event crítico en la tabla `launches`, y modificar `label_targets.py` para consultar el pool de PumpSwap al calcular `still_active_7d`.

FALLO C3 — Hermes Agent no existe como se describe

El blueprint asume la existencia de un agente Hermes con scheduler TOML nativo en producción. El repositorio real de NousResearch/hermes es un modelo fine-tuned LLM, no un framework de agentes con scheduler de tareas.

El comando 'hermes run --config config/hermes-memecoin.toml' del capítulo 9 fallará porque ese binario no existe. Cline intentará instalarlo y no encontrará nada funcional, bloqueando todo el despliegue.

Solución v2.3: reemplazar Hermes por APScheduler (Python puro) como scheduler de tareas. El LLM para generación de hipótesis se llama directamente vía LiteLLM Gateway (SAA v7.2 ya lo gestiona).

FALLO C4 — Helius free tier insuficiente para backfill

El free tier de Helius es ~100,000 requests/mes. El backfill de 90 días requiere: ~655,000 tokens/mes (dato real, paper Feb 2026) × 3 meses × ~4 llamadas RPC por token = ~7.8 millones de requests.

El backfill agotará el free tier de Helius en pocas horas, bloqueando todo el sistema durante semanas.

Solución v2.3: usar PumpPortal API + DexScreener como fuentes primarias del backfill (sin límite de rate o muy generosas), y Helius solo para enriquecer tokens individuales de alta prioridad.

FALLO C5 — Desbalance de clases extremo no gestionado

Un paper de febrero de 2026 (arxiv 2602.14860) analiza septiembre de 2025: de 655,770 tokens creados, solo 4,338 (0.63%) alcanzaron la graduación. Para pump ≥100% en 24h la ratio es aún menor.

El blueprint menciona pos_weight en XGBoost pero no especifica el valor ni el proceso de calibración. Con ratio 1:158, un pos_weight incorrecto hace que el modelo prediga siempre la clase mayoritaria (exactitud del 99.37%, pero precision@top10 = 0).

Solución v2.3: añadir calibración automática de pos_weight (ratio negativo/positivo), threshold tuning con curva precision-recall, y validación de que precision@top10 > 0.30 antes de aceptar el modelo.

2.2 IMPORTANTES — Degradan el rendimiento del modelo

#	Fallo	Impacto	Solución v2.3
I1	Wallet del creador ausente en features	Alto. Creadores con historial de rugs tienen 90%+ de probabilidad de rug en token nuevo. Feature más predictivo según literatura.	Añadir creator_rug_history_count y creator_total_tokens al schema y a token_features.

12	Bonding curve progress ignorado	Alto. El % de completitud de la bonding curve es un indicador directo de momentum y liquidez. DexScreener lo provee gratis.	Añadir bonding_curve_progress_pct (0-100%) a launches y token_features.
13	rugcheck.xyz API ignorada	Medio. API gratuita que calcula score de rug para cualquier token Solana. Puede ser feature directo sin entrenar modelo propio.	Llamar a rugcheck.xyz/tokens/{address}/report y guardar rugcheck_score en token_features.
14	Kaggle dataset no mencionado	Medio. Existe gleb270/pumpfun-api-solana-tokens-info en Kaggle. Acelera el backfill inicial.	Intentar descarga de Kaggle antes de Bitquery en el orden de fuentes del backfill.
15	Sin circuit breaker para APIs externas	Medio. Si Helius o Bitquery fallan, el backfill se rompe sin recovery automático.	Implementar tenacity con exponential backoff en todas las llamadas externas.
16	Directorio models/ no especificado	Bajo. Los modelos XGBoost serializados no tienen ruta definida en el proyecto.	Añadir models/{model_name}_{version}.pkl con registro en model_performance.

2.3 MENORES — Robustez y operación

#	Fallo	Solución rápida
M 1	blockSubscribe tiene problemas documentados de estabilidad en nodos ocupados	Usar logsSubscribe como método primario (más ligero) con blockSubscribe como fallback.
M 2	TimescaleDB requiere instalación compleja en macOS	Hacer TimescaleDB opcional: si no está disponible, usar PostgreSQL + índices parciales.
M 3	Telegram bot usa long polling (ineficiente)	Aceptable para MVP. Documentar migración a webhook como mejora futura.
M 4	El schema SQL no tiene índice en creator_address	Añadir CREATE INDEX idx_tokens_creator ON tokens(creator_address) para queries de historial de creador.
M 5	backtest_report.py no está especificado en detalle	Añadir estructura mínima: precision@top10 de últimas 24h + comparación con baseline naive.

3. INVESTIGACIÓN VALIDADA — Proyectos y Estrategias

3.1 Fuente de datos en tiempo real — PumpPortal WebSocket (GRATIS)

PumpPortal provee una API WebSocket oficial sin autenticación para eventos en tiempo real de Pump.fun:

```
uri = wss://pumpportal.fun/api/data
```

```
método subscribeNewToken → evento por cada token creado
```

```
método subscribeTokenTrade → trades en tiempo real por token
```

```
método subscribeMigration → migración a PumpSwap
```

Esta API es el método más simple y directo para producción. No requiere API key, no tiene límites documentados de rate, y los datos incluyen: mint address, name, symbol, bonding curve address, timestamp, creator, market cap inicial.

Fuente: pumpportal.fun/data-api/real-time/ — documentación oficial

3.2 Paper académico validado (febrero 2026) — Datos reales de Pump.fun

El paper 'Predicting the success of new crypto-tokens: the Pump.fun case' (arxiv 2602.14860, febrero 2026) analiza datos reales de septiembre de 2025:

- **Escala:** 655,770 tokens creados en 1 mes
- **Graduación:** Solo 4,338 tokens graduaron (0.63%) — confirma clase extremadamente desbalanceada
- **Creadores:** 243,123 wallets creadoras distintas — historial de creador es feature viable
- **Técnico:** Los tokens de Pump.fun son tokens de 6 decimales (no 9 como SPL estándar)

Implicación directa para v7.3: pos_weight óptimo para XGBoost Modelo A es aproximadamente 158 (ratio negativo/positivo).

3.3 Detección de rug pull con ML — Literatura científica

Dos papers relevantes de 2025 validan el enfoque del blueprint y añaden features no contemplados:

Paper 1 — 'Rug pull detection on DEX using transaction data' (Srifa et al., 2025):

- Ventanas temporales cortas (0-3h) son suficientes y más predictivas que ventanas largas

- Volume y transaction count son los features más importantes (top-2)
- Validador rewards (tips en Solana) son señal de actividad sospechosa

Paper 2 — 'Detecting Rug Pulls in DEX: The Rise of Meme Coins' (2025):

- 98% de tokens nuevos en DEX exhiben características fraudulentas (valida defense first)
- Contract code reuse por el creador es feature predictivo fuerte
- Features de BananaGun (herramienta de análisis) están en top-11 de importancia

3.4 Python + WebSocket — Implementación de referencia (Chainstack, código abierto)

El repositorio github.com/chainstacklabs/pumpfun-bonkfun-bot (MIT) proporciona implementación Python de referencia con:

- logsSubscribe como método principal (más estable que blockSubscribe)
- Geyser gRPC para latencia mínima en producción (Yellowstone)
- Decodificación del Anchor IDL de Pump.fun para extraer mint, bondingCurve, associatedBondingCurve
- Gestión de rate limits con max_rps y exponential backoff automático
- Detección de migración a PumpSwap vs Raydium por instrucción de programa

Cline puede usar este repositorio como referencia de código para collect_onchain.py.

3.5 rugcheck.xyz — API gratuita de scoring de rug

```
GET https://api.rugcheck.xyz/v1/tokens/{mint_address}/report
```

```
→ devuelve: score, risks[], top_holders, mint_authority, freeze_authority
```

Score de 0-1000 donde >500 = alto riesgo. Incluye análisis de: mint authority no revocada, freeze authority activa, concentración de holders. Feature gratuito que puede añadirse como columna rugcheck_score en token_features.

3.6 Bitquery — Datos históricos y streaming

Bitquery soporta múltiples modos de acceso para Pump.fun:

- GraphQL (free tier) para backfill histórico por rango de fechas
- WebSocket subscriptions para streaming en tiempo real
- Kafka/gRPC Shred Streams para latencia ultra-baja (plan de pago)

Para v7.3, el free tier de GraphQL es suficiente para el backfill. Los datos incluyen: creaciones, trades, OHLCV, bonding curve progress, top traders, y migración a PumpSwap.

4. PROMPT PARA VSCode CLINE — Genera SAA v7.3

El siguiente prompt está diseñado para ser pegado directamente en Cline (VSCode). Está estructurado para maximizar la implementación autónoma exitosa: primero establece el contexto del sistema SAA, luego las restricciones críticas, luego las tareas en orden de dependencias, y finaliza con los criterios de validación que Cline debe verificar automáticamente.

Estrategia de implementación para Cline: el prompt usa tareas numeradas con dependencias explícitas para que Cline pueda retomar desde cualquier punto sin perder contexto, y los criterios de éxito son comprobables mediante comandos bash sin intervención humana.

INSTRUCCIÓN DE USO: Copia todo el texto entre las líneas de separación (====) y pégalo como un único mensaje en Cline. No fragmentes el prompt.

PROMPT CLINE — MEMECOIN AGENT v7.3

Eres un desarrollador Python experto en blockchain y ML. Tu tarea es implementar el Sistema Autónomo de Análisis de Memecoins en Solana versión 7.3, basándote en el blueprint v2.2 con correcciones críticas aplicadas. **IMPORTANTE:** implementa en orden estricto de tareas. Tras cada tarea ejecuta los comandos de validación especificados antes de continuar.

CONTEXTO DEL SISTEMA (SAA v7.2)

El sistema corre sobre la infraestructura SAA v7.2 con los siguientes nodos Tailscale:

TO (100.68.1.180:8080) → LiteLLM Gateway (qwen3.5 / phi)

WS (100.68.1.160:11435) → llama-server Qwen3.5

IM (100.68.1.55:11434) → Ollama phi

MB / cliente Cline

El LLM endpoint es `http://100.68.1.180:8080/v1` (compatible OpenAI). Todo el código debe usar este endpoint para las llamadas a LLM, nunca APIs cloud externas de pago.

RESTRICCIONES CRÍTICAS (no negociables)

- NUNCA instalar ni referenciar 'hermes run' ni 'hermes agent'. No existe como binario. El scheduler se implementa con APScheduler (pip install apscheduler).
- NUNCA asumir que Helius free tier puede hacer el backfill completo. Usar PumpPortal API y DexScreener como fuentes primarias.
- NUNCA usar blockSubscribe como método principal WebSocket. Usar logsSubscribe.
- SIEMPRE hacer TimescaleDB opcional: si CREATE EXTENSION timescaledb falla, continuar con PostgreSQL puro y documentarlo en README.md.
- NUNCA ejecutar trades reales. EXECUTION_MODE=research es el valor por defecto inmutable en el .env generado.

TAREA 0 — Estructura del proyecto

Crea la estructura de directorios completa:

memecoin-agent/

├─ config/

| └─ .env.example

| └─ scheduler_config.py

├─ scripts/

| └─ backfill_historical.py

| └─ collect_onchain.py ← modo BACKFILL y modo LIVE

| └─ label_targets.py

| └─ compute_features.py

| └─ train_models_all.py

| └─ generate_hypotheses_llm.py

| └─ validate_hypotheses.py

| └─ backtest_report.py

| └─ telegram_bot.py

└─ sql/

```

├── schema_v7.3.sql
├── models/                                -- modelos XGBoost serializados aquí
├── logs/
├── Dockerfile
├── docker-compose.yml
├── docker-entrypoint.sh
├── requirements.txt
├── scheduler_main.py                    -- punto de entrada APScheduler
└── README.md

```

El archivo config/.env.example debe incluir TODAS las variables necesarias con valores de ejemplo. En especial:

```

HELIUS_API_KEY=your_helius_key
BITQUERY_API_KEY=your_bitquery_key
PUMPPORTAL_WS_URL=wss://pumpportal.fun/api/data
LITELLM_ENDPOINT=http://100.68.1.180:8080/v1
LITELLM_MODEL=qwen3.5
TELEGRAM_BOT_TOKEN=your_token
TELEGRAM_ALLOWED_USERS=your_user_id
EXECUTION_MODE=research
MAX_POSITION_SOL=0.5
RUGCHECK_API_BASE=https://api.rugcheck.xyz/v1
KAGGLE_USERNAME=optional
KAGGLE_KEY=optional

```

VALIDACIÓN TAREA 0:

```

ls -la memecoin-agent/scripts/ | wc -l    # debe ser >= 10
ls -la memecoin-agent/sql/
cat memecoin-agent/config/.env.example | grep EXECUTION_MODE    # debe mostrar research

```

TAREA 1 — Schema SQL v7.3

Crea sql/schema_v7.3.sql con las siguientes modificaciones respecto al blueprint v2.2:

Tabla tokens: añadir estos campos nuevos al esquema original:

```
-- Historial del creador (NUEVO en v7.3)

creator_rug_history_count    INTEGER DEFAULT 0,

creator_total_tokens_launched INTEGER DEFAULT 0,

creator_graduation_rate      FLOAT,


-- Bonding curve (NUEVO en v7.3)

bonding_curve_progress_pct   FLOAT,

pumpswap_pool_address        VARCHAR(255), -- relleno al migrar

migrated_to_pumpswap         BOOLEAN DEFAULT FALSE,


-- RugCheck score (NUEVO en v7.3)

rugcheck_score               INTEGER, -- 0-1000, >500 = alto riesgo

rugcheck_risks               JSONB;
```

Tabla launches: añadir:

```
bonding_curve_progress_pct   FLOAT, -- % completitud 0-100

rugcheck_fetched             BOOLEAN DEFAULT FALSE;
```

Tabla token_features: añadir:

```
-- Features de creador (NUEVO)

creator_rug_history_count    INTEGER,

creator_graduation_rate      FLOAT,


-- Bonding curve progress (NUEVO)

bonding_curve_progress_pct   FLOAT,


-- RugCheck (NUEVO)

rugcheck_score               INTEGER,


-- Índice adicional

CREATE INDEX idx_tokens_creator ON tokens(creator_address);
```

Añadir tabla nueva agent_config para parámetros de runtime:

```
CREATE TABLE agent_config (
    key    VARCHAR(100) PRIMARY KEY,
    value  TEXT NOT NULL,
    updated_at TIMESTAMP DEFAULT NOW()
);

INSERT INTO agent_config VALUES ('execution_mode', 'research', NOW());
```

VALIDACIÓN TAREA 1:

```
psql -d memecoin_db -c "\d tokens" | grep creator_rug
psql -d memecoin_db -c "\d token_features" | grep rugcheck
psql -d memecoin_db -c "SELECT * FROM agent_config;"
```

TAREA 2 — collect_onchain.py con modo LIVE WebSocket

Implementa collect_onchain.py con DOS modos de operación:

MODO BACKFILL (argumento `--mode backfill`): igual al blueprint v2.2, consulta HTTP periódica contra Helius/Bitquery para tokens históricos.

MODO LIVE (argumento `--mode live`, usado en producción): usar la siguiente estructura base con PumpPortal WebSocket:

```
import asyncio, websockets, json

from tenacity import retry, stop_after_attempt, wait_exponential

PUMPPORTAL_WS = os.getenv('PUMPPORTAL_WS_URL', 'wss://pumpportal.fun/api/data')

async def live_listener():
    async with websockets.connect(PUMPPORTAL_WS, ping_interval=20) as ws:
        await ws.send(json.dumps({'method': 'subscribeNewToken'}))
        await ws.send(json.dumps({'method': 'subscribeMigration'}))
        async for raw in ws:
            msg = json.loads(raw)
            if msg.get('txType') == 'create':
                await process_new_token(msg)
            elif msg.get('txType') == 'migrate':
```

```

        await process_migration(msg)

async def process_new_token(msg):

    # INSERT INTO tokens ON CONFLICT DO NOTHING

    # Lanzar coroutine de enriquecimiento async (rugcheck, creator history)

    pass

async def process_migration(msg):

    # UPDATE tokens SET migrated_to_pumpswap=TRUE,

    # pumpswap_pool_address=msg['pool'] WHERE address=msg['mint']

    pass

```

El modo live debe reconectarse automáticamente si el WebSocket se desconecta (bucle con `asyncio.sleep(5)` en el catch).

Para enriquecer cada token nuevo con rugcheck score, usar:

```

GET {RUGCHECK_API_BASE}/tokens/{mint}/report

→ token_features.rugcheck_score = data['score']

→ tokens.rugcheck_risks = json.dumps(data['risks'])

```

VALIDACIÓN TAREA 2:

```

python scripts/collect_onchain.py --mode live --dry-run --timeout 30

# Debe mostrar: 'WebSocket connected', al menos 1 token detectado en 30s, sin
excepciones

```

TAREA 3 — `backfill_historical.py` con fuentes corregidas

Implementa `backfill_historical.py` con el siguiente orden de prioridad de fuentes (crítico: diferente al `blueprint v2.2`):

```

PRIORIDAD 1: Kaggle dataset (si KAGGLE_USERNAME y KAGGLE_KEY están configurados)

→ from kaggle.api.kaggle_api_extended import KaggleApiClient

→ dataset: gleb270/pump-fun-api-solana-tokens-info

→ sin límites de rate, descarga masiva en parquet

PRIORIDAD 2: Bitquery GraphQL (free tier, sin key de pago)

→ query por rango de fechas contra programa
6EF8rrecthR5Dkzon8Nwu78hRvfCKubJl4M5uBEwF6P

```

```
→ paginación con cursor hasta cubrir 90 días
→ rate limit: max 10 req/min con tenacity backoff
```

PRIORIDAD 3: DexScreener REST (sin auth, generoso)

```
→ GET /latest/dex/tokens/{address} para precio y volumen
→ usar como enriquecimiento de precio para tokens ya descubiertos
```

PRIORIDAD 4: Helius RPC (solo para tokens individuales de alta prioridad)

```
→ getSignaturesForAddress con paginación 'before'
→ NUNCA para descubrimiento masivo (agota free tier)
```

Implementar checkpoint cada 500 tokens en backfill_log. Si el script se interrumpe, al reiniciar debe continuar desde el último checkpoint.

VALIDACIÓN TAREA 3:

```
python scripts/backfill_historical.py --days 7 --dry-run
# Debe mostrar: fuente seleccionada, estimación de tokens, sin errores
psql -d memecoin_db -c "SELECT data_source, COUNT(*) FROM tokens GROUP BY 1;"
```

TAREA 4 — label_targets.py con soporte PumpSwap

Implementa label_targets.py con la corrección crítica de PumpSwap para still_active_7d:

```
def label_still_active_7d(token):
    # Si migró a PumpSwap: consultar liquidez del pool de PumpSwap
    if token['migrated_to_pumpswap'] and token['pumpswap_pool_address']:
        pool_data = fetch_dexscreener(token['pumpswap_pool_address'])
        return pool_data['liquidity_usd'] > LIQUIDITY_MIN_THRESHOLD
    # Si no migró: criterio original sobre launches table
    else:
        return check_original_pump_fun_activity(token)
```

Regla adicional para tokens con datos incompletos (prevención survivorship bias, igual al blueprint):

```
# Token con < 2h de datos registrados → rug_pull_48h = True por defecto
if token_data_hours < 2:
    rug_pull_48h = True
```

```
label_source = 'insufficient_data_heuristic'
```

VALIDACIÓN TAREA 4:

```
python scripts/label_targets.py --token-limit 100 --dry-run
```

```
psql -d memecoin_db -c "SELECT pump_100pc_24h, rug_pull_48h, still_active_7d, COUNT(*)  
FROM tokens WHERE label_completed=TRUE GROUP BY 1,2,3 ORDER BY 4 DESC LIMIT 10;"
```

TAREA 5 — compute_features.py con features nuevos v7.3

Implementa compute_features.py añadiendo los nuevos features de v7.3:

```
# Features nuevos obligatorios en v7.3:
```

```
# 1. Historial del creador
```

```
creator_stats = get_creator_stats(token['creator_address'])
```

```
features['creator_rug_history_count'] = creator_stats['rug_count']
```

```
features['creator_graduation_rate'] = creator_stats['graduation_rate']
```

```
# 2. Bonding curve progress
```

```
features['bonding_curve_progress_pct'] = get_bonding_curve_pct(token)
```

```
# 3. RugCheck score (ya guardado en tokens.rugcheck_score)
```

```
features['rugcheck_score'] = token['rugcheck_score'] or -1 # -1 si no disponible
```

```
# 4. Feature version
```

```
features['feature_version'] = 'v1-onchain-v73' # nueva versión
```

La función get_creator_stats debe calcular desde la tabla tokens: cuántos tokens previos del mismo creator_address tienen rug_pull_48h=True, y la tasa de graduación.

VALIDACIÓN TAREA 5:

```
python scripts/compute_features.py --version v1-onchain-v73 --limit 50 --dry-run
```

```
psql -d memecoin_db -c "SELECT feature_version, COUNT(*), AVG(rugcheck_score) FROM  
token_features GROUP BY 1;"
```

TAREA 6 — train_models_all.py con calibración de desbalance

Implementa train_models_all.py con gestión automática del desbalance de clases:

```
from sklearn.utils.class_weight import compute_class_weight
```

```

import numpy as np

def train_model(X_train, y_train, model_name):

    # Calcular pos_weight automáticamente

    n_neg = (y_train == 0).sum()

    n_pos = (y_train == 1).sum()

    pos_weight = n_neg / max(n_pos, 1)  # nunca dividir por cero

    print(f'{model_name}: {n_pos} positivos, {n_neg} negativos,
    pos_weight={pos_weight:.1f}')

    model = XGBClassifier(

        n_estimators=300,

        max_depth=6,

        learning_rate=0.05,

        scale_pos_weight=pos_weight,

        eval_metric='aucpr',  # area bajo curva P-R, mejor que AUC-ROC para
desbalanceo

        early_stopping_rounds=30,

        random_state=42

    )

    # Threshold tuning: encontrar umbral que maximiza precision@top_k

    proba = model.predict_proba(X_test)[: , 1]

    threshold = find_optimal_threshold(proba, y_test, k=10)

    return model, threshold, pos_weight

```

GUARDAR modelos en models/{model_name}_{feature_version}_{date}.pkl

VALIDACIÓN TAREA 6:

```

python scripts/train_models_all.py --feature-version vl-onchain-v73

# Debe mostrar pos_weight para cada modelo

# precision@top_10 debe ser > 0.0 (si es exactamente 0.0, el modelo está roto)

ls -la models/

```



```
psql -d memecoin_db -c "SELECT model_name, precision_at_10, auc_roc FROM
model_performance ORDER BY created_at DESC LIMIT 6;"
```

TAREA 7 — scheduler_main.py con APScheduler

Implementa scheduler_main.py como reemplazo completo de Hermes. Este archivo es el punto de entrada del sistema:

```
from apscheduler.schedulers.blocking import BlockingScheduler

from apscheduler.triggers.cron import CronTrigger

from apscheduler.triggers.interval import IntervalTrigger

import subprocess, logging, os


scheduler = BlockingScheduler(timezone='UTC')

logger = logging.getLogger('memecoin_scheduler')


def run_script(script_name, args=''):

    cmd = f'python scripts/{script_name} {args}'

    try:

        result = subprocess.run(cmd.split(), capture_output=True, text=True,
                                timeout=600)

        if result.returncode != 0:

            logger.error(f'{script_name} FAILED: {result.stderr[:500]}')

        else:

            logger.info(f'{script_name} OK')

    except subprocess.TimeoutExpired:

        logger.error(f'{script_name} TIMEOUT after 600s')


# Tareas programadas

scheduler.add_job(lambda: run_script('collect_onchain.py', '--mode backfill'),

                  IntervalTrigger(minutes=15), id='collect')

scheduler.add_job(lambda: run_script('compute_features.py'),

                  IntervalTrigger(hours=1), id='features')

scheduler.add_job(lambda: run_script('label_targets.py'),

                  CronTrigger(minute=30), id='label')
```

```

scheduler.add_job(lambda: run_script('train_models_all.py'),
                  CronTrigger(hour=3), id='train')

scheduler.add_job(lambda: run_script('generate_hypotheses_llm.py'),
                  CronTrigger(day_of_week='mon', hour=4), id='hypotheses')

scheduler.add_job(lambda: run_script('validate_hypotheses.py'),
                  CronTrigger(hour='*/6'), id='validate')

scheduler.add_job(lambda: run_script('backtest_report.py'),
                  CronTrigger(hour=8), id='backtest')

# Iniciar live listener en thread separado
if os.getenv('LIVE_MODE', 'false') == 'true':
    import threading, asyncio

    from scripts.collect_onchain import live_listener

    thread = threading.Thread(target=lambda: asyncio.run(live_listener()),
                              daemon=True)

    thread.start()

scheduler.start()

```

VALIDACIÓN TAREA 7:

```

python scheduler_main.py --test-mode    # ejecuta cada tarea una vez y sale

# Debe completar sin excepciones

```

TAREA 8 — generate_hypotheses_llm.py con LiteLLM Gateway

Implementa generate_hypotheses_llm.py usando el LiteLLM Gateway del SAA v7.2:

```

import openai

client = openai.OpenAI(
    base_url=os.getenv('LITELLM_ENDPOINT'),
    api_key=os.getenv('LITELLM_API_KEY', 'dummy')
)

def generate_hypotheses():

```

```

# 1. Obtener top-20 winners y top-20 losers de la última semana

winners = get_top_tokens(target='pump_100pc_24h', limit=20)

losers = get_top_tokens(target='rug_pull_48h', limit=20)


prompt = f"""
Analiza estos datos on-chain de tokens de Pump.fun.

TOP 20 TOKENS CON PUMP >= 100% EN 24H:

{format_token_features(winners)}

TOP 20 TOKENS CON RUG PULL EN 48H:

{format_token_features(losers)}

Identifica MÁXIMO 5 hipótesis falsables y específicas que diferencian winners de
losers.

Cada hipótesis debe:

1. Hacer referencia a features on-chain concretos (valores numéricos)
2. Ser verificable con datos nuevos
3. NO ser una generalización obvia

Responde en JSON con estructura:

[{"hypothesis": "...", "feature": "...", "threshold": ..., "direction": ">"|"<"}]"""

response = client.chat.completions.create(
    model=os.getenv('LITELLM_MODEL', 'qwen3.5'),
    messages=[{'role': 'user', 'content': prompt}],
    temperature=0.3
)

return parse_hypotheses(response.choices[0].message.content)

```

VALIDACIÓN TAREA 8:

```

python scripts/generate_hypotheses_llm.py --dry-run

# Debe conectar al gateway SAA y retornar JSON válido

psql -d memecoin_db -c "SELECT COUNT(*) FROM token_hypotheses WHERE active=TRUE;"

```

TAREA 9 — telegram_bot.py completo

Implementa telegram_bot.py con todos los comandos del blueprint v2.2 MÁS estos nuevos de v7.3:

```
/top_rug_creators → top 10 creadores con más rugs históricos
/rugcheck {mint} → consulta rugcheck.xyz para un token específico
/creator {address} → historial de tokens del creador
/bonding {mint} → progreso actual de bonding curve
```

El decorador @restricted debe verificar contra TELEGRAM_ALLOWED_USERS del .env.

VALIDACIÓN TAREA 9:

```
python scripts/telegram_bot.py --test # verifica token y conexión sin iniciar polling
```

TAREA 10 — Docker Compose y requirements.txt

Implementa Docker Compose. El requirements.txt debe incluir EXACTAMENTE:

```
# Core
psycopg2-binary>=2.9
solana>=0.34
solders>=0.21
websockets>=12.0
requests>=2.31
asyncio-mqtt>=0.16 # opcional

# ML
xgboost>=2.0
scikit-learn>=1.4
pandas>=2.0
numpy>=1.26

# Scheduler
apscheduler>=3.10
```

```

# LLM

openai>=1.30


# Telegram

python-telegram-bot>=20.0


# Retry

tenacity>=8.2


# Data

pyarrow>=14.0      # para parquet del backfill
kaggle>=1.6        # opcional, para Kaggle backfill


# Opcional: Bitquery

gql>=3.4

requests-toolbelt>=1.0

```

El docker-entrypoint.sh debe:

1. Esperar a PostgreSQL (pg_isready loop)
2. Intentar CREATE EXTENSION IF NOT EXISTS timescaledb → si falla, loguear warning y continuar
3. Ejecutar psql -f sql/schema_v7.3.sql si tablas no existen
4. Si DB vacía: python scripts/backfill_historical.py --days 90
5. Lanzar en paralelo:
 - python scripts/telegram_bot.py &
 - python scheduler_main.py

VALIDACIÓN TAREA 10:

```

docker compose build 2>&1 | tail -5

docker compose up -d

sleep 30

docker compose ps    # todos los servicios deben estar healthy

docker compose logs agent 2>&1 | grep -E '(ERROR|OK|connected)' | head -20

```

TAREA 11 — CHECKLIST_PRE_PROD.md y README.md

Crea CHECKLIST_PRE_PROD.md con estos items adicionales respecto al blueprint v2.2:

```
[ ] PUMPPORTAL_WS_URL configurada y WebSocket live_listener conecta sin errores

[ ] PumpSwap migration events detectados (migrated_to_pumpswap = TRUE en algún token del backfill)

[ ] creator_rug_history_count calculado para al menos 100 tokens

[ ] rugcheck_score presente en al menos el 50% de token_features

[ ] pos_weight calculado automáticamente (no hardcodeado) en train_models_all.py

[ ] precision@top_10 > 0.0 para los 3 modelos (si es 0.0 el modelo está roto)

[ ] APScheduler corriendo y ejecutando todas las tareas sin errores

[ ] LiteLLM Gateway SAA responde en < 30s para generate_hypotheses_llm.py

[ ] EXECUTION_MODE=research en agent_config table de PostgreSQL

[ ] Docker Compose healthchecks pasando para todos los servicios
```

Crea README.md con secciones: Requisitos, Variables de entorno obligatorias, Inicio rápido Docker, Inicio nativo macOS, Comandos Telegram, Arquitectura resumida, Troubleshooting con los 5 errores más comunes.

VALIDACIÓN FINAL — ejecutar antes de marcar como completo:

```
cd memecoin-agent

# Test completo de integración (5 minutos)

python scripts/collect_onchain.py --mode live --dry-run --timeout 60 &

PID=$!

sleep 60

kill $PID

# Verificar tablas pobladas

psql -d memecoin_db -c "

SELECT

    (SELECT COUNT(*) FROM tokens) AS total_tokens,

    (SELECT COUNT(*) FROM tokens WHERE label_completed=TRUE) AS labeled,

    (SELECT COUNT(*) FROM token_features) AS features,

    (SELECT COUNT(*) FROM token_hypotheses WHERE active=TRUE) AS hypotheses,
```

```

(SELECT COUNT(*) FROM model_performance) AS model_runs
;"

# Verificar modo research activo

psql -d memecoin_db -c "SELECT value FROM agent_config WHERE key='execution_mode';"

# Debe retornar: research

```

Resumen de cambios v2.2 → v7.3

Cambio	v2.2 original	v7.3 corregido	Impacto
Data ingesta producción	Polling HTTP cada 15 min	PumpPortal WebSocket en tiempo real	CRÍTICO
Migración PumpSwap	No existe en el código	Detectado y etiquetado en label_targets.py	CRÍTICO
Scheduler	Hermes (no existe como binario)	APScheduler (Python puro)	CRÍTICO
Rate limits Helius	Sin gestión, agota free tier	Helius solo para tokens individuales	CRÍTICO
Desbalance de clases	pos_weight sin calibrar	pos_weight automático + threshold tuning	CRÍTICO
Feature creador	Ausente	creator_rug_history_count añadido	IMPORTANTE
Bonding curve progress	Ausente	bonding_curve_progress_pct añadido	IMPORTANTE
RugCheck API	Ausente	rugcheck_score como feature gratuito	IMPORTANTE
Kaggle dataset	No mencionado	Fuente prioritaria de backfill	IMPORTANTE
TimescaleDB	Obligatorio, falla en macOS	Opcional, fallback PostgreSQL	MENOR